

EXACT EXAM SOLUTION –

CIS-216L Student

Management CRUD

এই doc টা তোমার exact question paper অনুযায়ী
বানানো। ঠোঁট layer, ঠিক marks distribution অনুযায়ী।

Q1. Project Setup (5 marks)

start.spring.io → Maven, Java, Spring Boot latest

Dependencies: **Spring Web, Spring Data JPA,**

Thymeleaf, H2 Database, Lombok

Generate → Extract → VS Code এ Open Folder

Package বানাও (`src/main/java/com/example/demo/`
এর ভেতরে):

```
model  
repository  
service  
controller
```

(⚠️ এবার নতুন – `service` folder ও লাগবে)

Q2. Entity Class (5 marks) –

model/Student.java

```
package com.example.demo.model;

import jakarta.persistence.*;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;

@Entity
@Table(name = "students")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Student {

    @Id
    @GeneratedValue(strategy =
GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private String course;
    private Integer age;
}
```

Q3. Repository Layer (5 marks) –

repository/StudentRepository.java

```
package com.example.demo.repository;

import com.example.demo.model.Student;
import
org.springframework.data.jpa.repository.JpaRepository;
import
org.springframework.stereotype.Repository;

@Repository
public interface StudentRepository extends
JpaRepository<Student, Long> {
    // body ফাঁকা – save(), findAll(),
    findById(), deleteById() automatic
}
```

Q4. Service Layer (10 marks) –

service/StudentService.java

 এইটাই নতুন অংশ – Controller সরাসরি Repository কল করে না, মাঝে Service ক্লাস থাকে। কেন লাগে: business logic Controller থেকে আলাদা রাখার জন্য, এটা ভালো design practice।

```
package com.example.demo.service;

import com.example.demo.model.Student;
import
com.example.demo.repository.StudentRepository;
import
org.springframework.stereotype.Service;
import java.util.List;

@Service
public class StudentService {

    private final StudentRepository
studentRepository;

    public StudentService(StudentRepository
studentRepository) {
        this.studentRepository =
studentRepository;
    }

    // Save student
    public Student saveStudent(Student
student) {
        return
studentRepository.save(student);
    }

    // Get all students
    public List<Student> getAllStudents() {
        return studentRepository.findAll();
    }
}
```

```

        // Get student by ID
        public Student getStudentById(Long id)
        {
            return
            studentRepository.findById(id)
                .orElseThrow(() -> new
                RuntimeException("Student not found: " +
                id));
        }

        // Update student – save() নিজেই update
        করে যদি id থাকে
        public Student updateStudent(Student
        student) {
            return
            studentRepository.save(student);
        }

        // Delete student
        public void deleteStudent(Long id) {
            studentRepository.deleteById(id);
        }
    }

```

মনে রাখার ট্রিক: @Service annotation, constructor দিয়ে Repository inject করা – এই প্যাটার্ন Controller এর মতোই, শুধু @Controller এর বদলে @Service , আর এখানে কোনো URL mapping নাই, শুধু plain method।

Q5. Controller Layer (10 marks) –

controller/StudentController.java

```
package com.example.demo.controller;

import com.example.demo.model.Student;
import
com.example.demo.service.StudentService;
import
org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import
org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/students")
public class StudentController {

    private final StudentService
studentService;

    public StudentController(StudentService
studentService) {
        this.studentService =
studentService;
    }

    // /students → Show all students
    @GetMapping
    public String listStudents(Model model)
{
        model.addAttribute("studentList",
```

```
studentService.getAllStudents());
    return "student-list";
}

// /students/new → Show form
@GetMapping("/new")
public String showAddForm(Model model)
{
    model.addAttribute("student", new
Student());
    return "student-form";
}

// /students/save → Save student
@PostMapping("/save")
public String
saveStudent(@ModelAttribute Student
student) {

studentService.saveStudent(student);
    return "redirect:/students";
}

// /students/edit/{id} → Edit form
@GetMapping("/edit/{id}")
public String
showEditForm(@PathVariable Long id, Model
model) {
    model.addAttribute("student",
studentService.getStudentById(id));
    return "student-form";
}

// /students/delete/{id} → Delete
```

```

student
    @GetMapping("/delete/{id}")
    public String
deleteStudent(@PathVariable Long id) {
    studentService.deleteStudent(id);
    return "redirect:/students";
}
}

```

⚠ **খয়াল করো:** Controller এখন

`studentRepository` না, `studentService` inject
করছে। আর `@ModelAttribute Student student` কেই
দুইটা কাজে লাগানো যায় (Create + Update) – `save()`
নিজেই বুঝে নেয়।

Templates –

`src/main/resources/templates/`

`student-list.html`

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap
p@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
    <title>Students</title>
</head>

```



```

<body class="container mt-5">
  <h2>Student List</h2>
  <a th:href="@{/students/new}"
class="btn btn-success mb-3">Add New
Student</a>
  <table class="table table-striped">
    <thead class="table-dark">
      <tr><th>ID</th><th>Name</th>
<th>Email</th><th>Course</th><th>Age</th>
<th>Actions</th></tr>
    </thead>
    <tbody>
      <tr th:each="s :
${studentList}">
        <td th:text="${s.id}"></td>
        <td th:text="${s.name}">
</td>
        <td th:text="${s.email}">
</td>
        <td th:text="${s.course}">
</td>
        <td th:text="${s.age}">
</td>
        <td>
          <a
th:href="@{/students/edit/{id}
(id=${s.id})}" class="btn btn-sm btn-
warning">Edit</a>
          <a
th:href="@{/students/delete/{id}
(id=${s.id})}" class="btn btn-sm btn-
danger">Delete</a>
        </td>
      </tr>

```

```
        </tbody>
    </table>
</body>
</html>
```

student-form.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
    <title>Student Form</title>
</head>
<body class="container mt-5">
    <h2>Add / Edit Student</h2>
    <form th:action="@{/students/save}"
th:object="${student}" method="post">
        <input type="hidden" th:field="*
{id}">

        <div class="mb-3">
            <label class="form-
label">Name</label>
            <input type="text" th:field="*
{name}" class="form-control">
        </div>
        <div class="mb-3">
            <label class="form-
label">Email</label>
```

```
        <input type="email" th:field="*
{email}" class="form-control">
    </div>
    <div class="mb-3">
        <label class="form-
label">Course</label>
        <input type="text" th:field="*
{course}" class="form-control">
    </div>
    <div class="mb-3">
        <label class="form-
label">Age</label>
        <input type="number"
th:field="*{age}" class="form-control">
    </div>
    <button type="submit" class="btn
btn-primary">Save</button>
</form>
</body>
</html>
```

Q6. Test it UI (5 marks)

App run করো (`DemoApplication.java` → ▶),
browser এ যাও `http://localhost:8080/students` :

1. খালি table দেখবে
2. "Add New Student" → ফর্ম পূরণ করো (name, email, course, age) → Save
3. List এ নতুন student দেখাবে

4. Edit করে কিছু বদলাও → Save → update হবে
5. Delete করো → row চলে যাবে

আজকের Layer Order (মনে রাখার জন্য)

```
Entity → Repository → Service →  
Controller → Templates
```

নতুন জিনিস শুধু Service Layer। এইটা মনে রাখার সহজ উপায়:

- Repository: শুধু database এর সাথে কথা বলে (extends JpaRepository)
- Service: Repository কে call করে, business logic থাকে এখানে (@Service , constructor এ Repository ইনজেক্ট)
- Controller: Service কে call করে, web request handle করে (@Controller , constructor এ Service ইনজেক্ট)

তিনটাই একই প্যাটার্নে চলে: constructor দিয়ে নিচের layer ইনজেক্ট করা, তারপর তার method call করা।